ASSIGNMENT 7 / PIPELINE AUDIT

The Suborned

Sky Rumsey | Unity | Generating the city's people

The Suborned is a squad tactics game about using people's needs and relationships to carry out a plan. This pipeline creates the people who make that possible.

1. What it produces, and what still needs me

The pipeline creates **130 people, their daily routines and 222 relationship links**. In the playable mission, Inez the waitress owes money and is involved with Arthur, the door guard. Learning this gives the player a way to draw him away from his post.

An AI agent reads relevant game-design rules and proposes the cast. Code checks the proposal; a second agent reviews it. Once approved, the cast becomes people and routines in a format Unity can load **without hand reformatting**.

I still approve the cast and maintain the game rules. Publishing and first-play testing also need human attention. To automate more, I would connect the remaining release steps so one approved version is checked, packaged and uploaded together.

2. What I would change

I would save every input needed to rebuild a release. The replay exposed an old population file left over from before the city was redesigned. The four mission characters still match the shipped cast, but regenerating today changes some background residents and travel stops.

I would replace loose generated files with a versioned build record: the cast, city layout, rules, names and generator versions. A build would recreate anything affected by a change and stop if its inputs did not match the approved version.

3. What the generation actually cost

The recorded cast-generation run used about $0.69 across three AI calls, including the first rejected proposal. The final review was the largest call at **$0.28**. The writing agent used Claude Sonnet; the reviewer used Claude Opus.

These are Claude's reported usage costs. The calls were covered by a subscription, so this was not a separate API payment. Subscription fees, my time, hardware and hosting are excluded.

This is affordable for a solo developer at this scale: generation happens during development, approved content is reused, and rounds and spending are capped. **There is no AI inference cost during play.** Human review remains a significant time cost.

4. How I reduced wasted calls

Before: the AI reviewer ran even when code had already found a fault. In an earlier transit run, two such reviews used **$0.41**.

After: failed checks send the work straight back for correction. In the people run's failed first round, this meant **zero review calls, zero review tokens and $0 review cost**. The corrected result still received a full review. These are different candidates, so this is not a like-for-like percentage saving.

Recording limits: original AI responses are archived; the video records a real offline replay and the generated data running in an existing game executable. It does not show new AI calls or a newly compiled Unity build. The public download's two-minute first-play check is still outstanding.
Costs are rounded here; exact call records and verification are in the Assignment 7 evidence pack on GitHub.